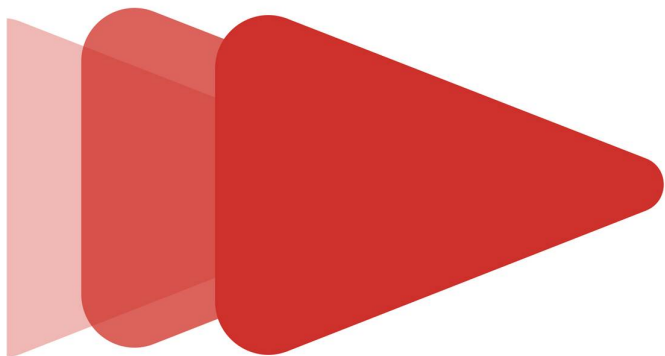




Informe del Primer Piloto CI/CD **Proceso DevSecOps** **Arquitectura Resiliente**

NAC-PTIIGE123-00000015

Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología

Noviembre 2023

Versión: 1.0.0

Firmas de Responsabilidad

Dirección Nacional de Tecnología		Fecha: noviembre 2023	VERSIÓN 1.0.0	PÁGINAS 42
NOMBRE Y CARGO			FIRMA	
APROBADO POR:	Ing. Paulina Correa A. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN- DATOS		 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
ELBORADO POR:	Ing. Christian Arreaga D. EXPERTO NACIONAL ARQUITECTURA SOLUCIONES TECNOLÓGICAS		 Firmado electrónicamente por: CHRISTIAN GUILLERMO ARREAGA DESIDERIO	
COLABORA:	Ing. Margarita Zapata EXPERTO NACIONAL DE DESARROLLO		 Firmado electrónicamente por: ESTHER MARGARITA ZAPATA GARCIA	
COLABORA:	Ing. Cristhian Cabezas G. EXPERTO NACIONAL DE DESARROLLO		 Firmado electrónicamente por: CRISTHIAN FABIAN CABEZAS GUTIERREZ	
COLABORA:	Ing. César Paullan P. ESPECIALISTA DE SEGURIDAD INFORMÁTICA		 Firmado electrónicamente por: CESAR AUGUSTO PAULLAN PILCO	
COLABORA:	Ing. Darwin Calle L. EXPERTO NACIONAL DE CONTROL Y ASEGURAMIENTO DE CALIDAD (QA)		 Firmado electrónicamente por: DARWIN AUGUSTO CALLE LOJA	
COLABORA:	Ing. David Mayorga P. EXPERTO NACIONAL DE SEGURIDAD INFORMÁTICA		 Firmado electrónicamente por: DAVID MAYORGA POLO	
COLABORA:	Ing. Jorge Armijo EXPERTO NACIONAL DE INFRAESTRUCTURA Y OPERACIONES, CENTRO DE COMPUTO		 Firmado electrónicamente por: JORGE LUIS ARMIGO QUITO	

Tabla de contenido

1. Introducción	4
1.1. Antecedentes	4
1.2. Alcance	5
1.3. Objetivos	5
2. Desarrollo	5
2.1. Marco Teórico	5
2.1.1. DevSecOps	5
2.1.2. Integración continua (CI)	6
2.1.3. Despliegue continuo (CD)	7
2.1.4. Herramientas colaborativas y tecnologías usadas en CI/CD	7
2.1.4.1. Git	7
2.1.4.2. GitLab	8
2.1.4.3. YAML	11
2.1.4.4. Docker	11
2.1.4.5. Kubernetes	12
2.1.4.6. SonarQube	13
2.1.4.7. Maven	15
2.1.4.8. Quarkus	16
2.1.4.9. Ansible	16
2.2. Desarrollo del piloto	17
2.2.1. Arquitectura conceptual propuesta	17
2.2.2. Recursos del piloto	18
2.2.3. Descripción del Piloto	19
2.2.4. Configuración del entorno de laboratorio	20
2.2.5. Despliegue del piloto	23
2.2.6. Definición y configuración de pipelines	24
2.2.7. Del flujo CI/CD	25
2.2.8. De Despliegue	31
2.2.8.1. Creación del Despliegue	33
2.2.8.2. Creación del Servicio	33

2.2.8.3.	Creación de la ruta Ingress.....	33
2.2.9.	Ejecución de pipelines.....	34
3.	Resultados del Piloto	36
3.1.	Resultados de la construcción del laboratorio del piloto.....	36
3.2.	Resultados del proceso CI/CD	37
4.	Conclusiones.....	37
4.1.	De parte de Desarrollo.....	37
4.2.	De parte de Seguridad Informática	37
4.3.	De parte de Infraestructura y Operaciones	38
4.4.	De Control de Calidad	38
4.5.	De parte de Planificación TI	38
5.	Recomendaciones.....	39
5.1.	De parte de Desarrollo.....	39
5.2.	De parte de Seguridad Informática	39
5.3.	De parte de Infraestructura y Operaciones	40
5.4.	De Control de Calidad	40
5.5.	De Planificación TI	41
6.	Glosario	42
7.	Bibliografía	42

1. Introducción

La Filosofía Institucional del Servicio de Rentas Internas, en su Visión plasma ser una Administración Tributaria reconocida por su permanente innovación y transformación digital, por tanto, para aportar a la visión institucional se plantea gestionar el ciclo de vida de las aplicaciones de la manera más efectiva, permitiendo a los equipos centrarse en su trabajo con calidad y eficiencia para brindar mejores resultados a los usuarios.

El Estatuto Orgánico de Gestión Organizacional por Procesos del Servicio de Rentas Internas, de acuerdo con su última reforma realizada mediante Resolución No. NAC-DGERCGC22-0000047 suscrito el 30 de septiembre de 2022 que rige desde el 1 de marzo de 2023, estipula entre los producto y servicios de la Dirección Nacional de Tecnología, el Entregable EOP: Tecnologías emergentes investigadas y propuestas.

La Gestión Interna del Departamento de Planificación TI y Gestión de Información-Datos siguiendo su cultura de investigación de tecnologías emergentes, identificó la necesidad de investigar la tecnología de contenedores de servicios para modernizar el despliegue de las aplicaciones bajo una arquitectura resiliente.

La implementación de DevSecOps con arquitecturas resilientes orientadas a contenedores implica la integración de los procesos de seguridad, desarrollo, pruebas, certificación y puesta en producción en una única cadena de herramientas integradas, esto permitirá que los Departamentos de Desarrollo, Seguridad Informática e Infraestructura y Operaciones trabajen en conjunto en un entorno colaborativo, acelerando el ciclo de vida del desarrollo con mayor seguridad y mejorando la calidad de las aplicaciones.

Para avanzar hacia esta implementación del proceso DevOps, en esta fase inicial, se realiza un primer piloto de CI/CD, para poder desplegar un microservicio en contenedores sobre una infraestructura propia de laboratorio y con ello poder obtener una visión más clara, así como también oportunidades de mejora.

1.1. Antecedentes

En el mercado, la industria tecnológica ha venido creando nuevas herramientas para modernizar la forma en la que se produce software, que implica cambiar muchas cosas. Estas herramientas permiten la integración a un esquema DevOps.

DevOps es una cultura que combina buenas prácticas y herramientas de software para mejorar el despliegue y puesta en marcha de aplicaciones de manera rápida y segura.

El Servicio de Rentas Internas tiene la necesidad de implementación y actualización de cambios para sus usuarios "Contribuyentes y Ciudadanos" de manera continua, por tanto, es fundamental modernizar la manera en la que se trabaja actualmente. Para esto se plantea la implementación de un proceso de Integración continua (CI) y Entrega continua (CD), y para

lograr este hito, es importante iniciar con ciertas actividades, que permitan visualizar donde estamos y a donde queremos llegar.

1.2. Alcance

Realizar una prueba de concepto inicial que permita determinar un flujo de trabajo de integración y entrega continua para los productos de software de la institución para dar paso a la creación del proceso de DevOps.

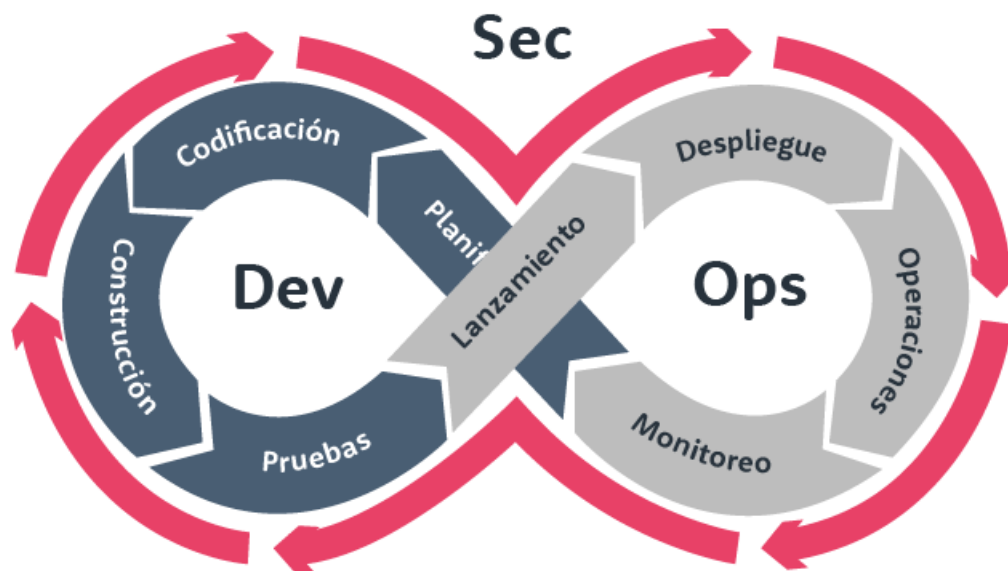
1.3. Objetivos

- Configurar un ambiente de laboratorio para implementar la prueba de concepto de DevSecOps.
- Integrar las herramientas actualmente utilizadas en la institución para diseñar el flujo inicial de CI/CD.
- Incluir herramientas adicionales que apoyen a la integración del flujo inicial de CI/CD.
- Diseñar los flujos de trabajo creando pipelines automatizados en GitLab para la integración y despliegue continuo de los aplicativos institucionales.
- Ejecutar el flujo inicial de CI/CD desde el desarrollo hasta su puesta en producción.

2. Desarrollo

2.1. Marco Teórico

2.1.1. DevSecOps



Su nombre, acrónimo de desarrollo, seguridad y operaciones, hace referencia a la integración de la seguridad en DevSecOps a través de la colaboración continua entre los equipos de desarrollo, operaciones y seguridad. En definitiva, es un término que refuerza la importancia de incorporar la seguridad al ciclo de vida de desarrollo de software. Es un método que se enmarca en el desarrollo ágil de software y supone una evolución hacia una responsabilidad compartida de la seguridad. Implica tener en cuenta la seguridad del

software desde el principio, así como automatizar procesos para no ralentizar el flujo de trabajo de DevOps. Su objetivo es conseguir entregas más rápidas y seguras de software, sin convertir la seguridad en un cuello de botella. Es decir, DevSecOps trata de garantizar una entrega rápida y segura resolviendo problemas habituales entre el desarrollo y la seguridad. Y, para ello, se apoya en herramientas. Si bien requiere también de una serie de cambios en la cultura de la organización para integrar la seguridad lo antes posible. Integrando la seguridad en la cultura, los procesos y las herramientas, DevSecOps garantiza que una entrega rápida de software no incluya vulnerabilidades de seguridad.

2.1.2. Integración continua (CI)

La Integración Continua (CI) es una práctica que incrementa la eficacia y la eficiencia de los resultados del equipo de desarrolladores. Se trata de combinar, de forma periódica y en un repositorio central, los cambios realizados en el código de un proyecto, para luego ejecutar pruebas y así detectar y reparar errores lo antes posible.

Esta metodología permite mejorar la calidad del código, entregar avances al cliente más a menudo y trabajar ágilmente con nuevos participantes del equipo, incluso si estos no conocen en profundidad el proyecto completo. “Detectar problemas de código es mucho más fácil con CI, evitando que estos errores lleguen a los clientes”.

Con la Integración Continua los desarrolladores integran frecuentemente su código a la rama principal de un repositorio común. En lugar de desarrollar partes del código de manera aislada e integrarlas al final del ciclo de producción, un desarrollador contribuirá con cambios a una parte del repositorio varias veces al día.

Beneficios de la Integración Continua

- Hace que el desarrollo de software sea más fácil, rápido y menos riesgoso para los desarrolladores. Al automatizar las compilaciones y las pruebas, los desarrolladores pueden realizar modificaciones más pequeñas y confirmarlas con confianza. Los desarrolladores reciben más rápido la retroalimentación sobre su código, aumentando el ritmo general de innovación.
- No existen tantos errores y se evita que estos se envíen a producción. Las pruebas automáticas captan las regresiones muy rápido. Permite que las correcciones se implementen más rápido, reduciendo el tiempo medio de resolución y el tiempo para restaurar el servicio.
- Desarrollar las versiones es más fácil. Los problemas que ocurren en la integración se solucionan mucho más rápido. Las pruebas automatizadas reducen en gran medida la cantidad de fallas que llegan a la operación.
- Las pruebas ya no son un gran costo para el equipo, pues la CI permite realizar muchas pruebas en cuestión de segundos.
- El control de calidad es mucho mejor debido a la automatización de pruebas.
- Al llegar a la operación menos errores, los usuarios quedan satisfechos y tienen una mejor experiencia, logrando métricas del servicio y generando nuevas preguntas.

2.1.3. Despliegue continuo (CD)

La entrega continua es en realidad una extensión de la Integración Continua, en la cual el proceso de entrega de software se automatiza para permitir implementaciones fáciles y confiables en la producción, en cualquier momento.

El objetivo de la entrega continua es mantener los conjuntos de cambios lo suficientemente pequeños para que ninguna actualización de la compilación principal comprometa el estado “listo para producción” del producto final si no está listo para su lanzamiento. El producto final puede tener errores menores, pero ninguno lo suficientemente importante como para comprometer la experiencia del usuario.

Un proceso de Entrega Continua (CD) bien implementado requiere una base de código que se pueda desplegar en cualquier momento. Con la entrega continua, los lanzamientos de nuevos cambios ocurren de manera frecuente y rutinaria.

Beneficios de la Entrega Continua

- Se mejora la productividad del equipo de desarrollo al liberarlos de las tareas manuales, pudiendo dedicar más tiempo a mejorar el nivel de servicio y agregar nuevas funcionalidades que generen valor empresarial.
- Un flujo de trabajo automatizado asegura que las tareas se completen siempre de la misma manera.
- Como la entrega continua permite realizar con facilidad pruebas más frecuentes y exhaustivas por haberse automatizado todo el proceso, los desarrolladores podrán descubrir y arreglar los errores antes que se conviertan en problemas más graves.
- Permite entregar actualizaciones con mayor rapidez y frecuencia, aumentando la confianza operativa, los niveles de servicio y el cumplimiento normativo, porque con un proceso de pruebas estandarizado cada vez se completan las tareas eficientemente.

2.1.4. Herramientas colaborativas y tecnologías usadas en CI/CD

Las herramientas colaborativas que se usaron en el piloto CI/CD, ya estaban siendo utilizadas en los diferentes ambientes de los departamentos de tecnología, con el objetivo de automatizar tareas y ejecutar pruebas de concepto.

2.1.4.1. Git

Constituye un sistema de control de versiones distribuido que posibilita la gestión eficiente de cambios en el código fuente durante el desarrollo de software. Permite la colaboración concurrente de múltiples desarrolladores en un proyecto, mitigando posibles conflictos y facilitando la administración de versiones.

Cuadro comparativo con otras alternativas:

Aspecto	Git	Mercurial	SVN (Subversion)
Tipo de Sistema	Distribuido	Distribuido	Centralizado
Rendimiento	Eficiente, permite operaciones locales rápidas.	Rápido y eficiente.	Más lento en operaciones distribuidas.
Curva de Aprendizaje	Pronunciada, especialmente para principiantes.	Menos empinada, interfaz más amigable.	Más fácil de aprender en comparación con Git.
Herramientas Gráficas	Variedad de interfaces gráficas disponibles.	Interfaz gráfica amigable.	Menos opciones de herramientas gráficas.
Gestión de Ramas	Fuerte y flexible, con ramas locales y remotas.	Buena gestión de ramas.	Menos flexible, más complicado de manejar.
Historial de Cambios	Robusto, registra cambios detalladamente.	Sólido historial de cambios.	Historial de cambios menos detallado.
Integración Continua	Soporte integrado y fácil de implementar.	Se integra bien con herramientas CI.	Requiere configuración adicional.
Herramientas de Colaboración	Depende de plataformas como GitLab o GitHub.	Integración con plataformas de colaboración.	Requiere herramientas externas para colaboración.
Popularidad	Altamente popular y ampliamente utilizado.	Menos popular en comparación con Git.	Ampliamente utilizado, pero disminuyendo.

2.1.4.2. GitLab

GitLab es una plataforma integral de gestión del ciclo de vida del desarrollo de software que ofrece soluciones completas desde la creación y seguimiento del código fuente hasta la entrega y despliegue de aplicaciones. GitLab puede entenderse como una herramienta que facilita y optimiza el proceso de desarrollo de software, garantizando una gestión eficiente y segura de los proyectos tecnológicos.

Para el SRI la adopción de GitLab en los procesos de tecnología, actúa como una "torre de control" centralizada para supervisar y coordinar todas las fases del desarrollo de software. Permite gestionar los proyectos de manera transparente, brindando una visión holística del progreso y las actividades de desarrollo. La plataforma simplifica la colaboración entre los equipos de desarrollo, control de calidad, operaciones y seguridad, promoviendo una comunicación fluida y una toma de decisiones informada.

Una de las principales fortalezas de GitLab desde una perspectiva gerencial es su capacidad para mejorar la seguridad de los sistemas desarrollados en el SRI. La plataforma integra funcionalidades de seguridad directamente en el proceso de desarrollo, identificando y corrigiendo posibles vulnerabilidades desde las primeras etapas del ciclo de vida del software. Esto no solo asegura aplicaciones más robustas, sino que también garantiza el

cumplimiento de estándares de seguridad, un aspecto crítico para una entidad gubernamental.

Desde el punto de vista operativo, GitLab optimiza la eficiencia al consolidar múltiples herramientas y procesos en una única plataforma. Facilita la gestión de configuraciones, automatiza tareas repetitivas y acelera la entrega de software. Esto resulta esencial para mantener el ritmo de las demandas cambiantes del SRI, permitiendo una evolución ágil y continua de los servicios tecnológicos.

Características técnicas del software GITLAB:

- GitLab se establece como una plataforma integral basada en web, orientada a la gestión de repositorios Git.
- Esta plataforma engloba funcionalidades como el seguimiento de problemas, integración continua y otros servicios relacionados.
- Proporciona una interfaz intuitiva que simplifica la coordinación y colaboración en proyectos de software, abarcando desde la planificación hasta la implementación.
- GitLab facilita la automatización de pruebas, compilación y despliegue, superando el desafío de integrar y desplegar cambios de manera rápida y eficiente en un entorno DevSecOps. Esto asegura una entrega continua y segura de software.
- La gestión eficiente de repositorios Git es esencial para un flujo de trabajo DevSecOps. GitLab ofrece herramientas sólidas para la gestión de ramas y fusiones, permitiendo un desarrollo paralelo y facilitando la colaboración entre equipos.
- GitLab incorpora funciones de análisis estático de código, identificando vulnerabilidades y errores en las fases iniciales del desarrollo. Esto aborda el desafío de garantizar la seguridad desde el inicio del ciclo de vida del software.
- La plataforma incluye herramientas integradas para el seguimiento de problemas y la gestión de proyectos, mejorando la visibilidad y colaboración entre equipos. Esto supera el desafío de coordinar actividades y mantener un seguimiento efectivo del progreso.
- GitLab facilita la colaboración en el código mediante revisiones integradas y comentarios, promoviendo una cultura de revisión por pares. Esto contribuye a la mejora continua del código y aborda el desafío de mantener altos estándares de calidad.
- Incorpora escaneo de seguridad de código directamente en el flujo de trabajo, permitiendo la detección temprana de vulnerabilidades. Este enfoque aborda el desafío de mantener la seguridad en un entorno DevSecOps de manera proactiva.
- GitLab ofrece capacidades de automatización y orquestación para tareas repetitivas y despliegues, lo que simplifica el proceso y supera el desafío de la gestión manual y propensa a errores.
- Al ser compatible con despliegues en la nube y on-premise, GitLab aborda el desafío de adaptarse a diferentes entornos tecnológicos, brindando flexibilidad a los equipos del SRI.

Cuadro comparativo con plataformas similares:

Aspecto	GitLab	GitHub	Bitbucket
Tipo de Plataforma	GitLab es tanto una plataforma de gestión de repositorios como una herramienta de CI/CD.	Principalmente una plataforma de alojamiento de repositorios Git.	Ofrece alojamiento de repositorios Git y soporte para Mercurial.
Integración Continua (CI/CD)	Integración CI/CD integrada y fácil de configurar.	CI/CD bien integrada con GitHub Actions.	Ofrece funcionalidades CI/CD con Pipelines.
Gestión de Problemas	Herramientas robustas para seguimiento de problemas y gestión de proyectos.	Ofrece seguimiento de problemas y gestión de proyectos con funcionalidades avanzadas.	Funcionalidades de seguimiento de problemas y gestión de proyectos.
Colaboración y Revisión de Código	Facilita la revisión de código y la colaboración en proyectos.	Herramientas sólidas para revisión de código y colaboración.	Permite la revisión de código y colaboración en el desarrollo.
Escalabilidad	Escalable para proyectos pequeños a grandes.	Escalabilidad para proyectos de cualquier tamaño.	Escalabilidad adecuada para diferentes tamaños de proyectos.
Nivel de Personalización	Ofrece un alto grado de personalización y configuración.	Menos opciones de personalización en comparación con GitLab.	Proporciona opciones de personalización, pero menos que GitLab.
Autenticación y Seguridad	Soporte para autenticación de dos factores y características de seguridad robustas.	Funciones de seguridad avanzadas y autenticación de dos factores.	Ofrece características de seguridad, pero menos opciones que GitLab.
Herramientas de Integración	Amplia gama de herramientas e integraciones disponibles.	Gran cantidad de integraciones disponibles.	Ofrece integraciones, pero con menos variedad.
Precio	GitLab ofrece versiones tanto gratuitas como de pago.	Ofrece opciones gratuitas y de pago.	Ofrece opciones gratuitas y de pago.

GitLab ofrece servicios tanto On-Premise como SaaS, por lo que a continuación, se describen ventajas y desventajas de estos 2 tipos de servicios:

Aspecto	On-Premise	SaaS
Ventajas	<i>Control Total:</i> La organización tiene control completo sobre la infraestructura y la seguridad.	<i>Escalabilidad:</i> Facilita la escalabilidad sin la necesidad de gestionar infraestructura local.
	<i>Personalización:</i> Permite ajustes y personalización según las necesidades específicas.	<i>Menores Costos Iniciales:</i> Requiere inversiones iniciales más bajas al evitar infraestructura local.
		<i>Colaboración Mejorada:</i> Proporciona una interfaz amigable y herramientas integradas para la colaboración.
		<i>Automatización:</i> Facilita la automatización de tareas mediante la integración continua.
Desventajas	<i>Costos Iniciales Elevados:</i> Implementar y mantener una infraestructura local puede ser costoso por contar con la infraestructura propia y personal con el conocimiento necesario.	<i>Dependencia de la Conectividad:</i> Requiere una conexión constante a Internet.
	<i>Actualizaciones y Mantenimiento:</i> La organización es responsable de las actualizaciones y el mantenimiento.	<i>Menos Control:</i> Cede parte del control sobre infraestructura y seguridad a los proveedores de servicios.

2.1.4.3. YAML

YAML es un lenguaje de declaración de datos que facilita la legibilidad y la capacidad de escritura. Este formato de serialización de datos se encarga de almacenar archivos de configuración y se puede usar junto con todos los lenguajes de programación.

Ventajas de YAML

- El uso de archivos con extensión *.yaml en el lenguaje YAML permite una mejor lectura de los datos, al tiempo que facilita su escritura. Así mismo, permite crear estructuras más complejas que las que se pueden hacer usando una línea de comandos.
- Además, YAML evita que el usuario tenga que especificar todos los parámetros en la línea de comandos. También presenta facilidad respecto a su mantenimiento, debido a que se puede administrar con un sistema de gestión de versiones, donde se registran los cambios y modificaciones realizadas.
- Otra de sus ventajas es que, como se ha mencionado anteriormente en este artículo, un archivo YAML admite un numeroso grupo de tipos de datos, como diccionarios, matrices, listas y escalares. Además, es compatible con lenguajes de programación como JavaScript, Python, Java o Ruby, entre otros.

2.1.4.4. Docker

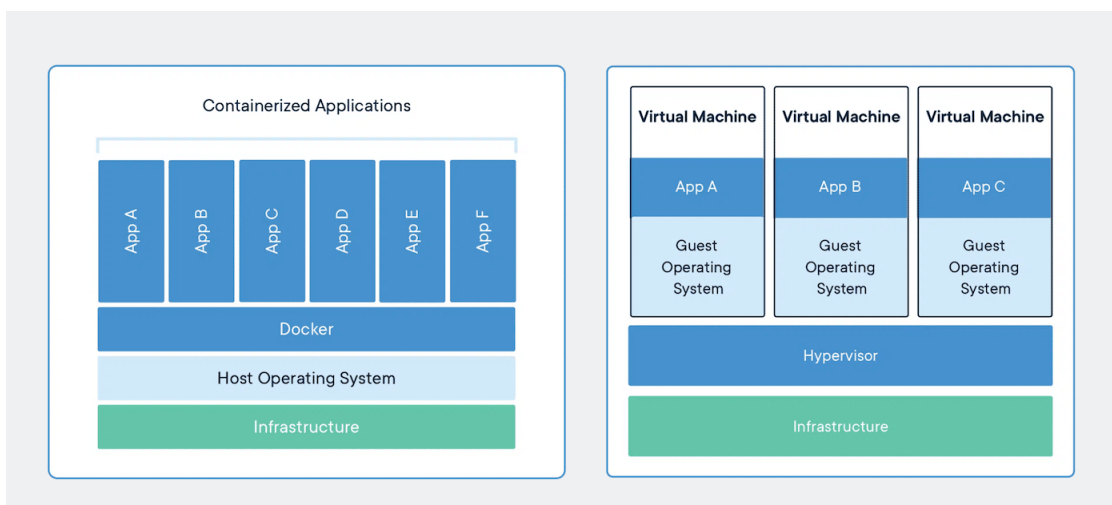
Es una plataforma de código abierto para desarrollar aplicaciones en un sandbox. Sus entornos virtuales ligeros se conocen como contenedores, los desarrolladores pueden

construir, probar y desplegar sus aplicaciones localmente o en un servidor de producción. En Docker, puedes empaquetar una aplicación en una unidad estandarizada, que puede utilizarse para el desarrollo de software. Esta unidad, o contenedor, incluye la codificación y las dependencias de la aplicación para que pueda ejecutarse fácilmente en cualquier entorno informático.

Antes de Docker, se solían utilizar máquinas virtuales (VM) para ejecutar aplicaciones. Éstas pueden emular ordenadores físicos, permitiendo a los desarrolladores convertir un servidor en varios. Sin embargo, este enfoque puede tener algunas desventajas.

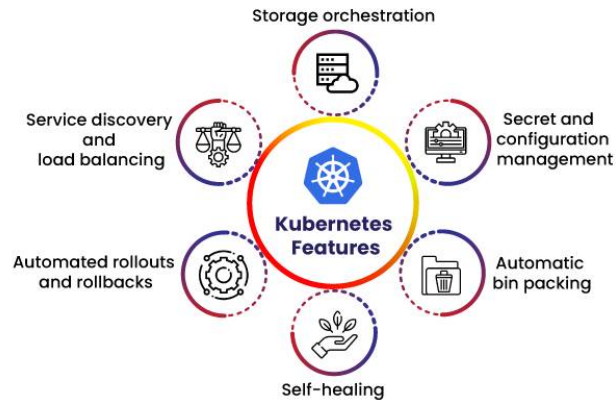
Cada máquina virtual contiene una copia completa del sistema operativo y de la aplicación, así como los binarios y bibliotecas necesarias. Estos archivos pueden ocupar decenas de GB en un ordenador. Además, la virtualización del hardware para un SO invitado puede requerir una sobrecarga considerable.

En lugar de virtualizar el hardware, los contenedores virtualizan el SO. En Docker, los contenedores son abstracciones en la capa de aplicaciones que pueden contener tanto código como dependencias. En la misma máquina, varios contenedores pueden ejecutarse como procesos aislados.



2.1.4.5. Kubernetes

Kubernetes o k8s es una plataforma de open source que automatiza las operaciones de un contenedor de Linux. Es un orquestador de contenedores, o lo que es lo mismo, el gestor del ciclo de vida de los contenedores de una aplicación, ya que ofrece servicios como el manejo del clúster (añadiendo o eliminando nodos), servicios de red y load-balancing (repartiendo la carga entre las distintas máquinas virtuales), servicios de monitorización o de checking del estado de salud de cada uno de los contenedores. Además, ofrece la opción del autoescalado y un Service Discovery para que un contenedor pueda encontrar con relativa facilidad las rutas IP o el DNS de otro contenedor.



2.1.4.6. SonarQube

SonarQube es una plataforma de código abierto para la inspección continua de la calidad del código a través de diferentes herramientas de análisis estático de código fuente. Proporciona métricas que ayudan a mejorar la calidad del código de un programa permitiendo a los equipos de desarrollo hacer seguimiento y detectar errores y vulnerabilidades de seguridad para mantener el código limpio.

Es una herramienta esencial para la fase de testing y auditoría de código dentro del ciclo de desarrollo de una aplicación y se considera perfecta para guiar a los equipos de desarrollo durante las revisiones de código. Soporta una etapa de inspección continua.

SonarQube está escrito en Java, pero puede analizar y administrar código 29 lenguajes de programación importantes, incluidos c / c ++, PL / SQL, Cobol, etc., a través de conjuntos de reglas incorporados que pueden ser ampliados con varios complementos.

Algunas características importantes tenemos las siguientes:

Característica	Descripción
Análisis Estático de Código	Realiza un análisis estático del código fuente para identificar y corregir problemas de calidad del código.
Detección de Defectos de Código	Identifica y señala defectos de código, incluyendo vulnerabilidades de seguridad, errores y malas prácticas.
Medición de la Deuda Técnica	Evalúa la deuda técnica asociada al código, ayudando a comprender y abordar los costos futuros de mantenimiento.
Cobertura de Código	Mide la cobertura del código mediante pruebas unitarias, identificando áreas del código que no están siendo probadas.

Análisis de Seguridad del Código	Realiza análisis específicos para identificar posibles vulnerabilidades de seguridad en el código fuente.
Integración con Herramientas de Construcción	Se integra con sistemas de construcción y automatización para proporcionar análisis continuo durante el desarrollo.
Soporte para Múltiples Lenguajes de Programación	Admite una amplia variedad de lenguajes de programación, permitiendo análisis en proyectos políglotas.
Informes y Paneles de Control	Proporciona informes detallados y paneles de control para monitorear y mejorar la calidad del código a lo largo del tiempo.
Integración con Flujos de Trabajo CI/CD	Se integra con flujos de trabajo de Integración Continua/Despliegue Continuo (CI/CD) para automatizar análisis y comentarios.

A continuación, se presenta un cuadro comparativo de la herramienta respecto de una similar en el mercado, con las siguientes características.

Característica Técnica	SonarQube	Veracode
Tipo de Herramienta	Análisis estático de código (SAST)	Prueba de seguridad de aplicaciones (AST)
Lenguajes de Programación Soportados	Amplia variedad de lenguajes (Java, C++, etc.)	Amplia variedad de lenguajes (incluyendo .NET)
Análisis Continuo	Sí	Sí
Detección de Vulnerabilidades	Sí (Identifica vulnerabilidades y errores)	Sí (Identifica vulnerabilidades y riesgos)
Integración CI/CD	Sí	Sí
Análisis de Seguridad del Código	Sí	Sí
Escaneo de Dependencias	Sí	No
Integración con IDEs	Sí	Sí
Gestión de Deuda Técnica	Sí	No
Cobertura de Código	Sí	No
Informes Detallados	Sí	Sí
Soporte para Desarrollo Políglota	Sí	Sí
Despliegue Local o en la Nube	Ambas opciones disponibles	Nube (Veracode as a Service)
Enfoque Principal	Calidad y seguridad del	Pruebas de seguridad de

	código	aplicaciones (AST)
--	--------	--------------------

Adoptar una herramienta como SonarQube en el proceso de **DevSecOps** para el Servicio de Rentas Internas (SRI) puede ofrecer una serie de beneficios significativos durante todo el ciclo del desarrollo del software, algunos de ellos se muestran a continuación:

- SonarQube realiza análisis estáticos continuos del código, identificando y señalando problemas de calidad, vulnerabilidades y errores. Esto contribuye a la mejora constante de la calidad del código, lo cual es crucial para un sistema como el SRI que maneja información sensible y operaciones críticas.
- Integra la seguridad directamente en el ciclo de vida del desarrollo, asegurando que las vulnerabilidades sean identificadas y abordadas desde las etapas iniciales del proceso de desarrollo. Esto es fundamental para garantizar la seguridad de las aplicaciones y sistemas del SRI.
- Al identificar de manera proactiva vulnerabilidades de seguridad y posibles riesgos, SonarQube permite a los equipos de desarrollo y seguridad abordar problemas antes de que se conviertan en amenazas críticas. Esto contribuye a una gestión más efectiva de los riesgos de seguridad.
- Ayuda en el cumplimiento de normativas y estándares de seguridad, lo cual es esencial para una entidad gubernamental como el SRI. Facilita la identificación y corrección de posibles violaciones normativas en el código fuente.
- Al automatizar análisis y proporcionar informes detallados, SonarQube ayuda a los equipos de desarrollo a ser más eficientes. La detección temprana de problemas permite una resolución más rápida, reduciendo el tiempo y los costos asociados con la corrección de errores más adelante en el ciclo de desarrollo.
- Ofrece informes detallados y paneles de control que proporcionan visibilidad completa sobre la calidad y la seguridad del código. Esto facilita la toma de decisiones informadas y transparentes en todos los niveles del desarrollo y operación.
- Fomenta una cultura de colaboración entre equipos de desarrollo, operaciones, calidad y seguridad al proporcionar una plataforma centralizada para la revisión y mejora continua del código. Facilita la comunicación y la colaboración entre diferentes partes del proceso DevSecOps.

2.1.4.7. Maven

Apache Maven es una herramienta que estandariza la configuración de un proyecto en todo su ciclo de vida, como por ejemplo en todas las fases de compilación y empaquetado y la instalación de mecanismos de distribución de librerías, para que puedan ser utilizadas por otros desarrolladores y equipos de desarrollo.

Principales características de Maven

Básicamente, Apache Maven no deja de ser la base de los compiladores actuales, de IDEs como Eclipse, NetBeans o IntelliJ, a los que ofrece soporte gracias a algunas de sus características, como, por ejemplo:

- Un sistema de gestión dependencias.

- Un mecanismo distribuido de distribución de librerías. El comportamiento distribuido es siempre desde el repositorio local de Maven hacia los repositorios que están publicados en Internet o en la red corporativa.
- Mecanismos para ser extensible, por la creación de plugins personalizables.
- Es multi-plataforma, puede funcionar tanto en entornos Linux como Windows al ser una aplicación Java.
- Es software libre, con lo cual es el código está disponible, se podría modificar y customizar en caso de que fuera necesario.
- Fomenta la reutilización de código y de librerías. El hecho de que Apache Maven ofrezca repositorios oficiales y públicos de software libre, con librerías desplegadas, que toda la comunidad de desarrolladores de software utiliza, hace que este concepto también pueda trasladarse al mundo empresarial, a través de repositorios remotos corporativos, compartidos por distintos equipos de proyectos o el propio equipo de desarrollo.
- Es compatible con múltiples IDEs.

2.1.4.8. Quarkus

Quarkus es un entorno desarrollado por RedHat para crear aplicaciones Java. Quarkus se desarrolló con el objetivo de ejecutar programas Java en contenedores. En particular, se centra en el apoyo al software de configuración Kubernetes. Otro enfoque del desarrollo de Quarkus es el uso de bibliotecas y estándares Java establecidos.

“HotSpot”, del proyecto OpenJDK, se utiliza como máquina virtual Java (JVM) para que sea la capa de ejecución del código Java. Además, también se puede utilizar el desarrollo “GraalVM”, que se basa en HotSpot. Este último permite compilar el código Java en código máquina directamente ejecutable. Para entender el beneficio inmediato de utilizar Quarkus, veamos primero cómo se ejecutan las aplicaciones Java con y sin Quarkus.

2.1.4.9. Ansible

Ansible es una plataforma de automatización de código abierto que puede utilizarse para gestionar grandes grupos de sistemas informáticos. Te ayuda a automatizar el despliegue de aplicaciones, la gestión de la configuración, el aprovisionamiento en la nube, la actualización de estaciones de trabajo y servidores, entre otras muchas tareas.

Beneficios de Ansible

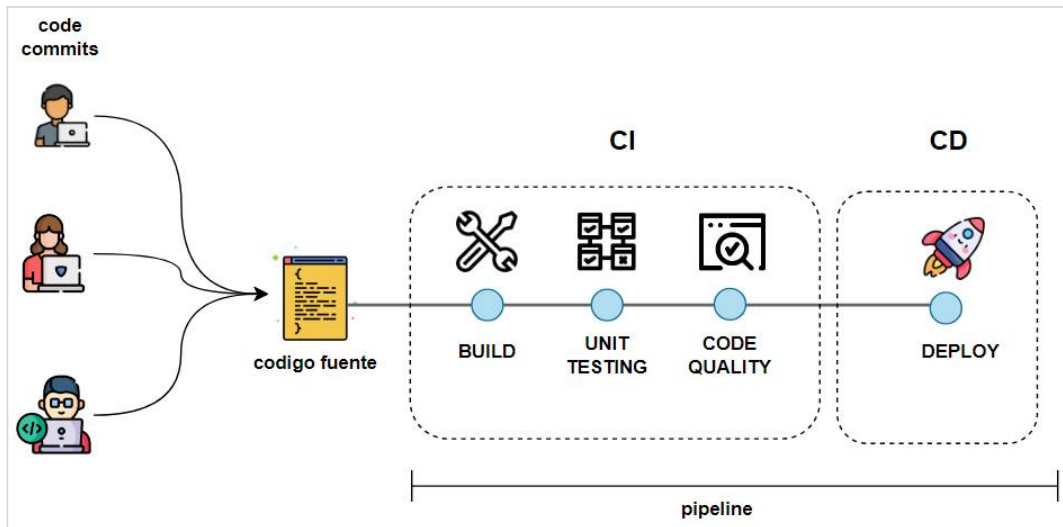
- Ansible proporciona a los ingenieros de DevOps cuatro beneficios principales:
- Reduce los recursos necesarios para la gestión IT: los Sysadmins pueden contener cientos o incluso miles de máquinas desde un único punto y de una sola vez.
- Hace accesible la automatización: uno de los objetivos de Ansible fue que se requiriera un aprendizaje mínimo de uso. La plataforma utiliza YAML (Yet Another Markup Language), un lenguaje legible para personas, con elementos de otros idiomas de programación.
- No afecta al rendimiento: no requiere agentes, ni software, ni recursos informáticos que se ejecute o instale en sistemas gestionados.

- Garantiza la coherencia: la plataforma se diseñó para ser mínima y permitir a los usuarios crear entornos coherentes. Además, como toda la operación se realiza a través de una conexión SSH, no añade más complejidad a los sistemas.

2.2. Desarrollo del piloto

2.2.1. Arquitectura conceptual propuesta

Para el primer piloto se contempló la siguiente arquitectura conceptual:

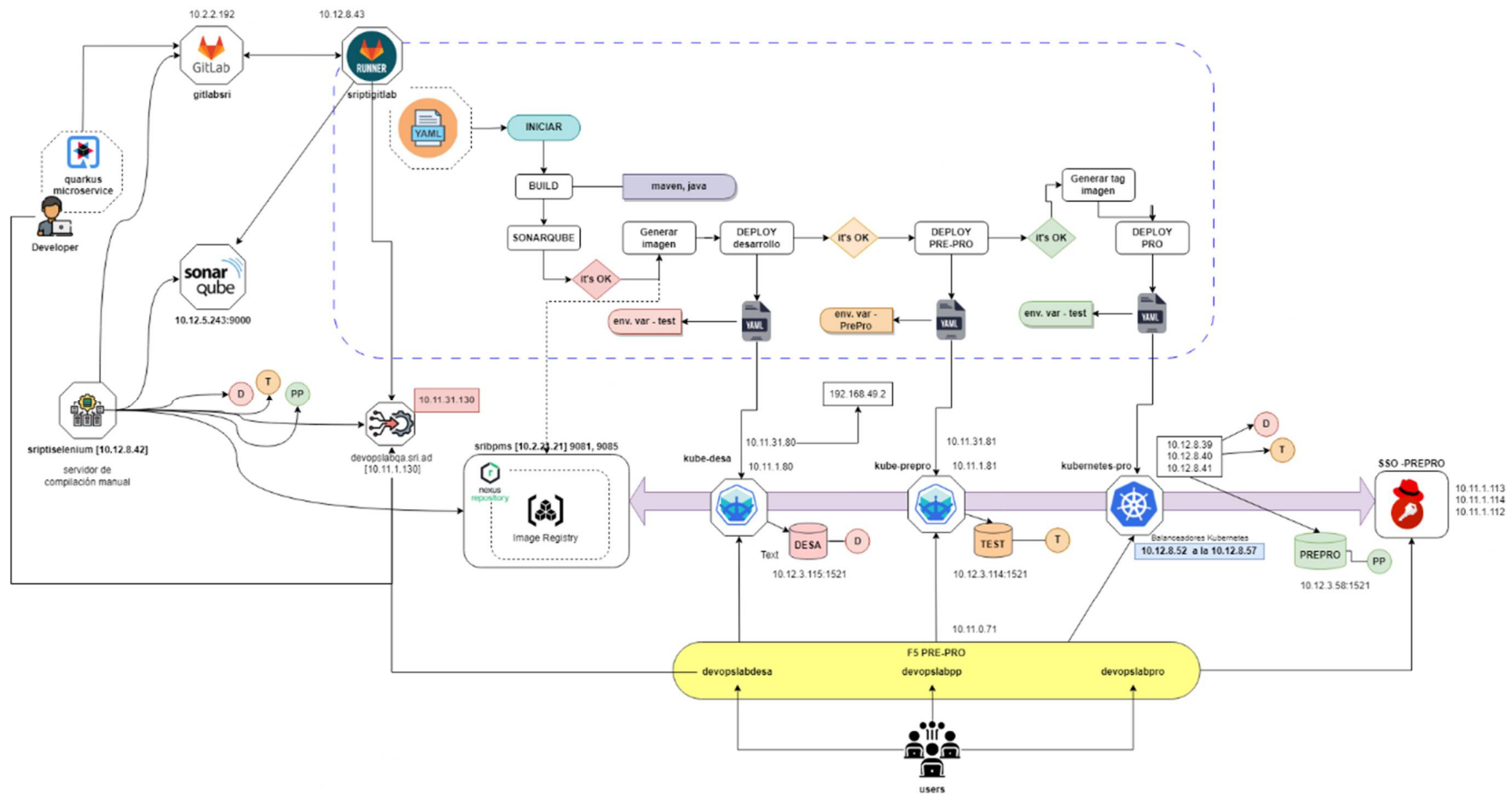


El diagrama anterior muestra las fases que se han tomado en cuenta para el piloto, a continuación, se describen los componentes:

Componente	Descripción
Code commits	Los diferentes contribuidores o desarrolladores del código fuente realizan commits al repositorio de código basado en GIT.
Código fuente	El código fuente se encuentra centralizado en una plataforma basada en Git como por ejemplo GitLab.
Pipeline	La configuración del conjunto de pasos que involucran las actividades que se ejecutan en cada fase se encuentran definidos en un archivo yaml que representa el pipeline del piloto.
Build	En esta etapa se ejecutan las tareas de compilación y construcción del aplicativo.
Unit Testing	En esta etapa se ejecutan las pruebas unitarias del aplicativo usando herramientas por ejemplo Junit.
Code Quality	En esta etapa se realiza el análisis de calidad del código usando herramientas como por ejemplo SonarQube
Deploy	En esta etapa se realiza el despliegue con la finalidad que se realicen las primeras validaciones del funcionamiento.

2.2.2. Recursos del piloto

Para la implementación del laboratorio que se usó en la construcción del primer piloto de CI/CD, se contó con los siguientes recursos los cuales fueron configurados e integrados:



Cantidad	Descripción	Previamente existente en ambiente de:
1 servidor	GitLab Plataforma basada en GIT	Desarrollo
1 servidor	GitLab Runner	Planificación TI
1 servidor	Para SonarQube	Control de Calidad
1 servidor	Para Nexus OSS (Docker Registry)	Planificación TI
1 servidor	Para plataforma de despliegue del ambiente simulado de Desarrollo (Minikube)	
1 servidor	Para plataforma de despliegue del ambiente simulado de Pre-Producción (Minikube)	
3 servidores	Para clúster Kubernetes para el ambiente simulado de Producción (Kubernetes)	Planificación TI
1 servidor	Para un servidor SSO	Desarrollo
1 balanceador Capa 3	F5	Redes
1 instancia de base de datos	Base de datos INTRASRI	Desarrollo
1 instancia de base de datos	Base de datos INTRASRI	Control de Calidad - Testing
1 instancia de base de datos	Base de datos INTRASRI	Control de Calidad – Pre Producción
1 servidor	Para compilación manual	Planificación TI
1 servidor	Para levantar servicios utilitarios	Control de Calidad
3 servidores	Para un clúster transversal de Red Hat SSO	Control de Calidad – Pre Producción

Conforme a los descrito en la tabla anterior algunos recursos han sido reutilizados del ambiente de Control de Calidad, Desarrollo y Planificación TI.

2.2.3. Descripción del Piloto

Para el presente piloto se dividió su ejecución en dos fases la de configuración del entorno del laboratorio y el despliegue de este para luego continuar con la prueba mediante la ejecución de los pipelines.

2.2.4. Configuración del entorno de laboratorio

En esta fase se procede a describir los entornos configurados para el desarrollo del piloto, se mencionan las herramientas utilizadas y las configuraciones que han sido necesarias aplicar.

- **Asunciones iniciales:** El laboratorio se configuró bajo las siguientes premisas:
 - ♦ La aplicación utilizada en el piloto fue el proyecto de la prueba de concepto de microservicio denominado sri-autorizacion-api ubicado en el repositorio GitLab (<https://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api.git>)
 - ♦ La navegación de todos los servicios que se expongan mediante web para este primer piloto se lo realizó mediante el protocolo HTTP puerto 80.
 - ♦ Se reutilizó todos los componentes existentes en la infraestructura establecida dentro de los diferentes ambientes tecnológicos.
- **Servidores:** Los servidores utilizados cuentan con un sistema operativo Red Hat Enterprise Linux 8.x y poseen usuarios con permiso sudo para la ejecución de los comandos por consola.
- **Kubernetes:** Con la finalidad de simular un ambiente de producción se instaló un clúster de Kubernetes versión 1.25.1 usando 3 servidores con la configuración de básica de 1 master o control plane y 2 nodos, adicional se configuraron herramientas adicionales tales como:
 - Calico (Network Addon)
 - Metal LB (Load Balancer)
 - Ingress
 - Helm

Para su configuración se utilizaron los siguientes tutoriales:

<https://www.linuxtechi.com/how-to-install-kubernetes-cluster-rhel/>
<https://computingforgeeks.com/deploy-metallb-load-balancer-on-kubernetes/>
<https://www.adaltas.com/en/2022/09/08/kubernetes-metallb-nginx/>

- **Minikube:** De igual manera para la simulación de los ambientes de despliegue de Desarrollo y Pre-Producción se instaló **Minikube versión 1.31.2**. Algunas referencias que se siguieron para la instalación son las siguientes:
 - ♦ <https://minikube.sigs.k8s.io/docs/start/>
 - ♦ <https://www.fosstechnix.com/how-to-install-minikube-on-centos-8/>
 - ♦ <https://www.learnitguide.net/2023/04/how-to-install-minikube-on-centos-8.html>
- **Apache-Proxy Reverso:** Para la redirección del tráfico entre el clúster minikube (192.168.60.2) y el equipo anfitrión hacia el exterior se utilizó Apache Web Server módulo proxy-reverso. A continuación, algunos recursos de donde se tomó de referencia para su implementación:
 - ♦ <https://httpd.apache.org/>
 - ♦ https://httpd.apache.org/docs/trunk/es/howto/reverse_proxy.html
- **Configuración de Dominios - Exposición de Servicios – Balanceo Carga:** Con el objetivo de poder simular un entorno más real en la prueba de concepto inicial, se elaboraron las

siguientes configuraciones a nivel del F5 de contingencia (GYE) para la exposición de los servicios RESTful por cada uno de los entornos que la iniciativa persigue, los cuales se detallan a continuación:

- ♦ **devopslabdesa.sri.gob.ec:** Dominio de pruebas que tiene como objetivo ser la puerta de entrada para la validación de los servicios expuestos en el proceso de desarrollo y pruebas de la coordinación de Desarrollo e Implementación. Este dominio estará disponible en: <http://devopslabdesa.sri.gob.ec/server-status>
- ♦ **devopslabpp.sri.gob.ec:** Dominio de pruebas que tiene como objetivo exponer los servicios en un ambiente de preproducción (Ambientes de Control de Calidad), para posterior coordinar los tipos de Pruebas Funcionales y No Funcionales de las aplicaciones, servicios o componentes de software. Este dominio de pruebas estará disponible en: <http://devopslabpp.sri.gob.ec/server-status>
- ♦ **devopslabpro.sri.gob.ec :** Este dominio de pruebas se lo creo con el objetivo de simular un punto de entrada para las pruebas finales de las aplicaciones, servicios o componentes de software que cumplieron todas las fases del ciclo del desarrollo del software hasta su puesta en producción. Este dominio estará disponible en: <http://devopslabpro.sri.gob.ec/server-status>
- **Git y GitLab:** Se utilizó Git como sistema de control de versiones en su versión 2.31.1 y GitLab Community Edition en su versión 15.4.1 como plataforma basada en Git para el alojamiento del código fuente del proyecto sri-autorizacion-api, gestión de versiones y DevOps. Adicional se configuró un GitLab Runner en los servidores con la finalidad de usarlo exclusivamente para las ejecuciones de los pipelines de este piloto.

Para su configuración se utilizó el siguiente tutorial:

<https://unixcop.com/install-gitlab-on-centos-8-stream/>

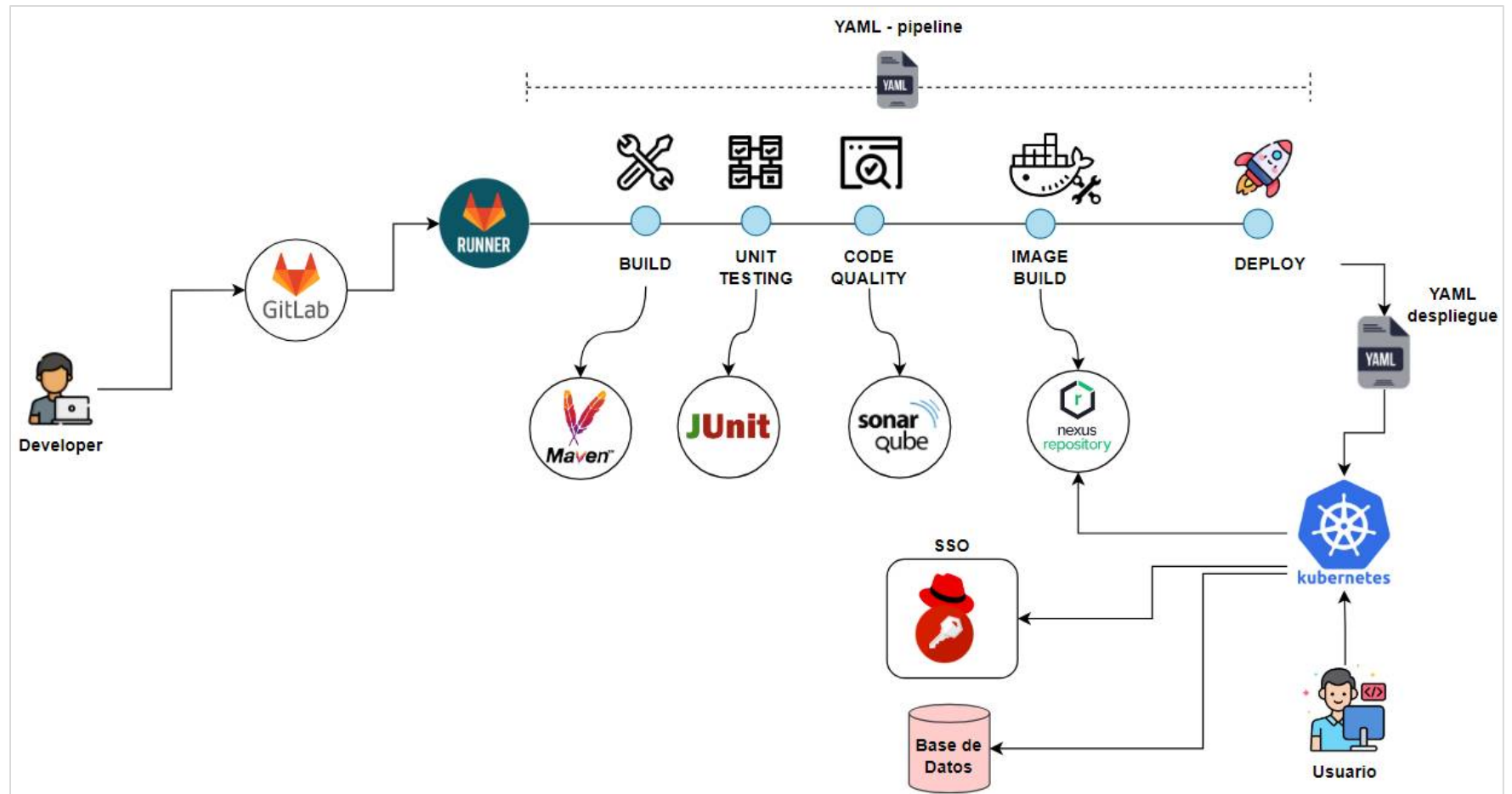
- **Nexus:** Se realizó la instalación de Sonatype Nexus OSS versión 3 con la finalidad de alojar un registro de imágenes Docker privado. Se siguió el siguiente tutorial para su configuración: <https://devopscube.com/how-to-install-latest-sonatype-nexus-3-on-linux/>.
- **Entorno de compilación:** El entorno de compilación fue instalado en el mismo servidor GitLab runner ya que esta fase es parte de la ejecución del pipeline, para lo cual se instalaron las siguientes herramientas:
 - Podman versión 4.1.1
 - Apache Maven versión 3.9.4
 - OpenJDK versión 17.0.8

Adicional se configuro el acceso remote al clúster Kubernetes con la finalidad de realizar el despliegue a través del pipeline.

- **Dominios de prueba y balanceador:** Se solicitó la creación de los siguientes dominios:

- ✓ devopslabdesa.sri.gob.ec. - Para el ambiente simulado de Desarrollo
 - ✓ devopslabpp.sri.gob.ec. - Para el ambiente simulado de Preproducción.
 - ✓ devopslabpro.sri.gob.ec. - Para el ambiente simulado de Producción.
- **Acceso al Private Docker Registry (Nexus):** Para obtener acceso al repositorio de imágenes Docker almacenado en Nexus fue necesario realizar lo siguiente:
- ✓ Autenticar Podman con el registro mediante el siguiente comando:
podman login 10.2.21.21:9081
- Las credenciales utilizadas fueron:
Usuario: admin
Password: N3xUz2022
- ✓ Configurar la URL del repositorio de imágenes como inseguro debido a la navegación HTTP. Esto se lo realizó editando el archivo /etc/containers/registries.conf añadiendo la siguiente configuración a la sección "registry"
- ```
[[registry]]
location = "10.2.21.21:9085"
insecure = true
```
- **Permisos de Firewall.** – Acorde a los lineamientos comunicados por el área de Seguridad Informática se solicitaron los permisos de firewall conforme al proceso establecido y acorde las necesidades que se fueron identificando para la comunicación de los equipos y los servicios que conforman el primer piloto CI/CD desplegados en el laboratorio.

## 2.2.5. Despliegue del piloto





Para el despliegue de este primer piloto de CI/CD se tomó como premisa la configuración y ejecución de un flujo básico donde un aplicativo que contiene una implementación de un API pase desde la fase de desarrollo hasta su despliegue en una plataforma de contenedores para que pueda ser accesible para su consumo. Cabe mencionar que, aunque el laboratorio contempla más componentes que los que describe el despliegue estos no pudieron integrarse al mismo debido a inconvenientes de conectividad asociados a permisos de firewall, infraestructura de redes y desconocimiento de las nuevas herramientas, los mismos podrán ser incluidos en futuros pilotos ya que actualmente se encuentran en su mayoría configurados y son parte del laboratorio.

### 2.2.6. Definición y configuración de pipelines

Todos los proyectos tienen una serie de requisitos para ser ejecutados. Ya sean librerías o programas instalados en el propio entorno, ya sean archivos de configuración específicos. Nuestro objetivo es automatizar la creación de un entorno idóneo para la ejecución de nuestro proyecto y después automatizar la ejecución del proyecto en sí. Para conseguir esto, debemos definir varias tareas, como la instalación de dependencias o librerías, los comandos necesarios para ejecutar nuestro proyecto, etc.

Todas estas tareas las vamos a separar en «etapas» o «stages». Las etapas son agrupaciones de tareas y tienen un orden específico (definido por nosotros). Cada etapa puede tener una o más de una tarea y las tareas de las etapas no se ejecutan hasta que todas las tareas de la etapa anterior hayan finalizado su ejecución con éxito, a menos que nosotros especifiquemos lo contrario. Un pipeline es la manera abreviada de referirse a dicho cúmulo de etapas en un proyecto.

Para nuestro proyecto definimos 3 etapas o stages:

- ✓ primerafase
- ✓ segundafase
- ✓ tercerafase

Donde la primera y segunda fase definirá el CI del proyecto con la construcción, pruebas y despliegue en el repositorio.

Como tercera fase contempla el despliegue del proyecto en los diferentes ambientes.

Se configura las siguientes variables para las ejecuciones.

```
JAVA_HOME : "/opt/SDKMAN/candidates/java/current"
M2_HOME : "/opt/SDKMAN/candidates/maven/current"
SONAR_HOME: /home/jboss/herramientas/sonar-scanner-4.8.0.2856-linux_desa
M2_HOME_SONAR: "/opt/SDKMAN/candidates/maven/apache-maven-3.9.4_sonar"
JAVA_HOME_SONAR: "/opt/SDKMAN/candidates/java/17.0.8-graal"
SONAR_HOME: "/opt/SDKMAN/candidates/sonar/sonar-scanner-4.8.0.2856-linux_desa"
```

Y las siguientes variables globales:

CI\_REGISTRY: Ip del repositorio de imágenes (register)  
 CI\_REGISTRY\_PASSWORD: clave del repositorio de imágenes (register)  
 CI\_REGISTRY\_USER: usuario del repositorio de imágenes (register)  
 SONAR\_HOST\_URL: url sonarqube  
 SONAR\_TOKEN: token sonarqube

### 2.2.7. Del flujo CI/CD

```
image: busybox:latest

stages:
 - primerafase #1
 - segundafase
 - tercerafase

#PARA GENERAR ARTEFACTOS Y REVISION DE SONAR

variables:
 JAVA_HOME : "/opt/SDKMAN/candidates/java/current"
 M2_HOME : "/opt/SDKMAN/candidates/maven/current"
 SONAR_HOME: /home/jboss/herramientas/sonar-scanner-4.8.0.2856-linux_desa
 M2_HOME_SONAR : "/opt/SDKMAN/candidates/maven/apache-maven-3.9.4_sonar"
 JAVA_HOME_SONAR : "/opt/SDKMAN/candidates/java/17.0.8-graal"
 SONAR_HOME : "/opt/SDKMAN/candidates/sonar/sonar-scanner-4.8.0.2856-linux_desa"

cache:
 key: "${CI_JOB_NAME}"
 paths:
 - .sonar/cache

##Branch
generar-snapshot:
 stage: primerafase
 script:
 - export JAVA_HOME=$JAVA_HOME
 - export M2_HOME=$M2_HOME
 - export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
 - mvn package -Dmaven.test.skip=true
 except:
 - master
 tags:
 - run_devops

generar-sonarqube:
 stage: primerafase
 script:
```

```
- export JAVA_HOME=$JAVA_HOME_SONAR
- export M2_HOME=$M2_HOME_SONAR
- export SONAR_HOME=$SONAR_HOME
- export PATH=$JAVA_HOME/bin:$SONAR_HOME/bin:$M2_HOME/bin:$PATH
- mvn package -Dmaven.test.skip=true
- sonar-scanner -Dproject.settings=sonar-scanner.properties -
Dsonar.coverage.jacoco.xmlReportPaths=report-aggregate/target/site/jacoco-aggregate/jacoco.xml
dependencies:
- generar-snapshot
except:
- master
tags:
- run_devops
```

```
crear-imagen:
stage: segundafase
script:
- export JAVA_HOME=$JAVA_HOME
- export M2_HOME=$M2_HOME
- export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
- mvn package -Dmaven.test.skip=true
- podman build -f src/main/docker/Dockerfile.jvm -t sri-autorizacion-api-jvm .
dependencies:
- generar-sonarqube
except:
- master
tags:
- run_devops
tag-imagen:
stage: segundafase
script:
- podman image tag sri-autorizacion-api-jvm:latest 10.2.21.21:9085/qa/sri-autorizacion-api-
jvm:latest
- podman image push 10.2.21.21:9085/qa/sri-autorizacion-api-jvm:latest
except:
- master
tags:
- run_devops
ejecutar-imagen:
stage: tercerafase
script:
- kubectl delete -f sri-autorizacion-api-jvm-pti.yaml
- kubectl apply -f sri-autorizacion-api-jvm-pti.yaml
except:
- master
tags:
- run_devops
```

```
#1 MASTER
configurar-variables:
 stage: primerafase
 script:
 - echo "The job's stage is '$CI_JOB_STAGE'"
 only:
 - master
 tags:
 - run_devops

generar-snapshot:
 stage: primerafase
 script:
 - export JAVA_HOME=$JAVA_HOME
 - export M2_HOME=$M2_HOME
 - export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
 - mvn package -Dmaven.test.skip=true
 only:
 - master
 tags:
 - run_devops
```

```
generar-sonarqube:
 stage: primerafase
 script:
 - export JAVA_HOME=$JAVA_HOME_SONAR
 - export M2_HOME=$M2_HOME_SONAR
 - export SONAR_HOME=$SONAR_HOME
 - export PATH=$JAVA_HOME/bin:$SONAR_HOME/bin:$M2_HOME/bin:$PATH
 - mvn package -Dmaven.test.skip=true
 - sonar-scanner -Dproject.settings=sonar-scanner.properties -
Dsonar.coverage.jacoco.xmlReportPaths=report-aggregate/target/site/jacoco-aggregate/jacoco.xml
 dependencies:
 - generar-snapshot
 only:
 - master
 tags:
 - run_devops

crear-imagen:
 stage: segundafase
 script:
 - export JAVA_HOME=$JAVA_HOME
 - export M2_HOME=$M2_HOME
 - export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
 - mvn package -Dmaven.test.skip=true
 - podman build -f src/main/docker/Dockerfile.jvm -t sri-autorizacion-api-jvm .
```

```
dependencies:
 - generar-sonarqube
only:
 - master
tags:
 - run_devops
```

```
tag-imagen:
 stage: segundafase
 script:
 - podman image tag sri-autorizacion-api-jvm:latest 10.2.21.21:9085/qa/sri-autorizacion-api-jvm:latest
 - podman login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
 - podman image push 10.2.21.21:9085/qa/sri-autorizacion-api-jvm:latest
 dependencies:
 - generar-sonarqube
 only:
 - master
 tags:
 - run_devops
```

```
ejecutar-imagen:
 stage: tercerafase
 script:
 - kubectl delete -f sri-autorizacion-api-jvm-pti.yaml
 - kubectl apply -f sri-autorizacion-api-jvm-pti.yaml
 dependencies:
 - generar-sonarqube
 only:
 - master
 tags:
 - run_devops
```

```
pruebas-humo:
 stage: tercerafase
 script:
 - cd /home/labqa/python
 - python3 test7.py
 dependencies:
 - generar-sonarqube
 only:
 - master
 tags:
 - run_devops
```



La primera fase contempla dos tareas:

Generar-snapshot: Es el encargado de compilar el proyecto

```

56 [INFO] --- quarkus:2.11.2.Final:build (default) @ sri-autorizacion-api ---
57 [INFO] [org.hibernate.Version] HHH000412: Hibernate ORM core version 5.6.10.Final
58 [INFO] [io.quarkus.deployment.QuarkusAugmentor] Quarkus augmentation completed in 6169ms
59 [INFO] -----
60 [INFO] BUILD SUCCESS
61 [INFO] -----
62 [INFO] Total time: 14.867 s
63 [INFO] Finished at: 2023-11-27T12:47:00-05:00
64 [INFO] -----
65 Saving cache for successful job 00:00
66 Creating cache generar-snapshot-1-protected...
67 Runtime platform arch=amd64 os=linux pid=2925550 revision=d89a789a version=16.4.1
68 WARNING: .sonar/cache: no matching files. Ensure that the artifact path is relative to the working directory (/home/
69 labqa/builds/H377cMw6/0/Arquetipos/piloto-base-microservicios/sri-autorizacion-api)
70 Archive is up to date!
71 Created cache
72 Cleaning up project directory and file based variables 00:00
73 Job succeeded

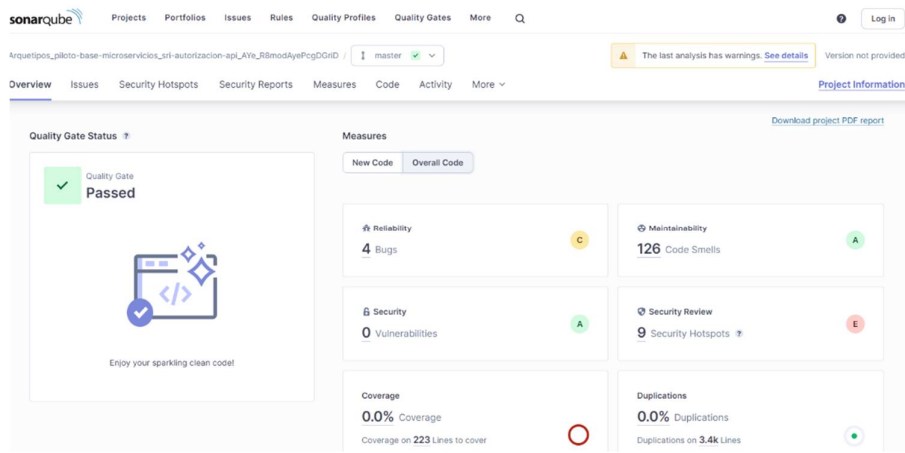
```

Generar-sonarqube: Esta tarea nos ayuda a ejecutar sonarqube generando un informe de las posibles vulnerabilidades de código

```

375 INFO: QUALITY GATE STATUS: PASSED - View details on http://10.12.5.243:9000/dashboard?id=Arquetipos_piloto-base-microservicios_sri-autorizacion-api_AYe_R8modAyePcg06riD5branch=master
376 INFO: Analysis total time: 2:50.894 s
377 INFO: -----
378 INFO: EXECUTION SUCCESS
379 INFO: -----
380 INFO: Total time: 2:54.085s
381 INFO: Final Memory: 75M/254M
382 INFO: -----
383 Saving cache for successful job 00:00
384 Creating cache generar-sonarqube-1-protected...
385 Runtime platform arch=amd64 os=linux pid=1943381 revision=d89a789a version=16.4.1
386 WARNING: .sonar/cache: no matching files. Ensure that the artifact path is relative to the working directory (/home/
387 labqa/builds/H377cMw6/0/Arquetipos/piloto-base-microservicios/sri-autorizacion-api)
388 Archive is up to date!
389 Created cache
390 Cleaning up project directory and file based variables 00:01
391 Job succeeded

```



La segunda fase contempla dos tareas:

Crear-imagen: Construye la imagen en nuestro contenedor podman

```
85 STEP 10/11: ENV JAVA_OPTS="-Dquarkus.http.host=0.0.0.0 -Djava.util.logging.manager=org.jboss.logmanager.LogManager"
86 --> cc067f359d7
87 STEP 11/11: ENV JAVA_APP_JAR="/deployments/quarkus-run.jar"
88 COMMIT sri-autorizacion-api-jvm
89 --> 00789e39181
90 Successfully tagged localhost/sri-autorizacion-api-jvm:latest
91 00789e39181becfc037351bce8c0de49349c68fb260400134dfeffb8ac333b95
92 Saving cache for successful job
93 Creating cache create-image-1-protected...
94 Runtime platform arch=amd64 os=linux pid=1945049 revision=d89a789a version=16.4.1
95 WARNING: .sonar/cache: no matching files. Ensure that the artifact path is relative to the working directory (/home/labqa/builds/M377cMw6/0/Arquetipos/piloto-base-microservicios/sri-autorizacion-api)
96 Archive is up to date!
97 Created cache
98 Cleaning up project directory and file based variables
99 Job succeeded
```

Tag-imagen: Es el encargado de añadir el nombre de la imagen y subirlo en el repositorio

```

20 Successfully extracted cache
21 Executing "step_script" stage of the job script
22 $ podman image tag sri-autorizacion-api-jvm:latest 10.2.21.21:9085/qa/sri-autorizacion-api-jvm:latest
23 $ podman login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
24 Login Succeeded!
25 $ podman image push 10.2.21.21:9085/qa/sri-autorizacion-api-jvm:latest
26 Getting image source signatures
27 Copying blob sha256:bf4887e46ac204643afc8748d83bb1f9dabc366da839d8a1df7c3f9c58c4e8fd
28 Copying blob sha256:dcf8ecdb306c8961a5ed624a23dd232787d2f3a77ebb87e977d747838a86d329
29 Copying blob sha256:ec19ba8366c5473f468cb622d55b024edd3a0434bde38b0620985be9ccc64ea5
30 Copying blob sha256:1fa0ae62c0a06f2c0ebb6270c81ff459761734f0915ba2c5049f7246a743014
31 Copying blob sha256:6ded8e1287680a279ef68ea6dd02344e462218baf9f0dc7d483ed643db38396d
32 Copying blob sha256:b73e456bd68ff923538c7a871e884594938ee763dc62f674ca624bd5ac4fa158
33 Copying blob sha256:b1d592a149a879a139dd0c401eca998a35ae8ebf1a15cf93e367e65e9fd2fd84
34 Copying config sha256:80789e39181becfc837351bce8c8de49349c68fb260480134dfeffb8ac333b95
35 Writing manifest to image destination
36 Storing signatures
37 Saving cache for successful job
38 Creating cache tag-imagen-1-protected...
39 Runtime platform arch=amd64 os=linux pid=1945651 revision=d89a789a version=
16.4.1
40 WARNING: .sonar/cache: no matching files. Ensure that the artifact path is relative to the working directory
(/home/labqa/builds/M377cMw6/0/Arquetipos/piloto-base-microservicios/sri-autorizacion-api)
41 Archive is up to date!
42 Created cache
43 Cleaning up project directory and file based variables
44 Job succeeded

```

Finalmente, la tercera fase contempla el despliegue en nuestro entorno de kubernetes

```

18 No URL provided, cache will not be downloaded from shared cache server. Instead a local version of cache will
be extracted.
19 Successfully extracted cache
20 Executing "step_script" stage of the job script
21 $ kubectl delete -f sri-autorizacion-api-jvm-pti.yaml
22 deployment.apps "sri-autorizacion-api-jvm-pti" deleted
23 service "sri-autorizacion-api-jvm-pti" deleted
24 $ kubectl apply -f sri-autorizacion-api-jvm-pti.yaml
25 deployment.apps/sri-autorizacion-api-jvm-pti created
26 service/sri-autorizacion-api-jvm-pti created
27 Saving cache for successful job
28 Creating cache ejecutar-imagen-1-protected...
29 Runtime platform arch=amd64 os=linux pid=1946258 revision=d89a789a version=
16.4.1
30 WARNING: .sonar/cache: no matching files. Ensure that the artifact path is relative to the working directory
(/home/labqa/builds/M377cMw6/0/Arquetipos/piloto-base-microservicios/sri-autorizacion-api)
31 Archive is up to date!
32 Created cache
33 Cleaning up project directory and file based variables
34 Job succeeded

```

## 2.2.8. De Despliegue

A continuación, se muestra el manifiesto YAML utilizado para el despliegue de la aplicación de prueba, el mismo contiene la creación del despliegue, servicio y ruta de Ingress.

```

apiVersion: apps/v1
kind: Deployment
metadata:
 labels:
 app.kubernetes.io/version: 1.0.0-SNAPSHOT
 app.kubernetes.io/name: sri-autorizacion-api-jvm-p1
 name: sri-autorizacion-api-jvm-p1
spec:

```



```

replicas: 1
selector:
 matchLabels:
 app.kubernetes.io/version: 1.0.0-SNAPSHOT
 app.kubernetes.io/name: sri-autorizacion-api-jvm-p1
template:
 metadata:
 labels:
 app.kubernetes.io/version: 1.0.0-SNAPSHOT
 app.kubernetes.io/name: sri-autorizacion-api-jvm-p1
 spec:
 imagePullSecrets:
 - name: nexus-pti-secret
 hostAliases:
 - hostnames:
 - "sri51"
 ip: "10.12.3.115"
 containers:
 - image: 10.2.21.21:9085/sri-autorizacion-api-jvm:latest
 imagePullPolicy: Always
 name: sri-autorizacion-api-jvm-p1
 ports:
 - containerPort: 8580
 name: http
 protocol: TCP

apiVersion: v1
kind: Service
metadata:
 labels:
 app.kubernetes.io/name: sri-autorizacion-api-jvm-p1
 app.kubernetes.io/version: 1.0.0-SNAPSHOT
 name: sri-autorizacion-api-jvm-p1
spec:
 ports:
 - name: http
 port: 80
 protocol: TCP
 targetPort: 8580
 selector:
 app.kubernetes.io/name: sri-autorizacion-api-jvm-p1
 app.kubernetes.io/version: 1.0.0-SNAPSHOT

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: sri-autorizacion-api-jvm-p1
spec:
 ingressClassName: nginx

```

```

rules:
- host: kubelab.sri.gob.ec
 http:
 paths:
 - path: /autorizacion/
 pathType: Prefix
 backend:
 service:
 name: sri-autorizacion-api-jvm-p1
 port:
 number: 80

```

### 2.2.8.1. Creación del Despliegue

Para el despliegue podemos resaltar la configuración realizada en lo referente al uso del Docker Registry privado para lo cual se definieron los siguientes parámetros:

#### Credenciales del Docker Registry privado:

imagePullSecrets:

- name: nexus-pti-secret

#### Definir la imagen a utilizar:

containers:

- image: 10.2.21.21:9085/sri-autorizacion-api-jvm:latest

#### Exponer el puerto del aplicativo:

ports:

- containerPort: 8580
  - name: http
  - protocol: TCP

### 2.2.8.2. Creación del Servicio

Para la creación del servicio se configuro tomando en cuenta lo siguiente:

Tipo del servicio: ClusterIP (si se omite la configuración lo toma por defecto)

Para exponer el puerto del servicio:

ports:

- name: http
  - port: 80
  - protocol: TCP
  - targetPort: 8580

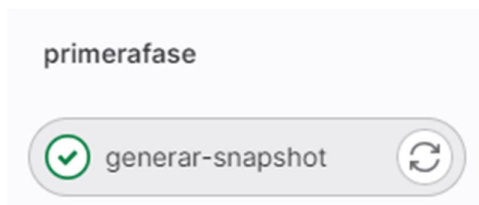
### 2.2.8.3. Creación de la ruta Ingress

Para la exposición del servicio a través del controlador Ingress se realizó la siguiente configuración:

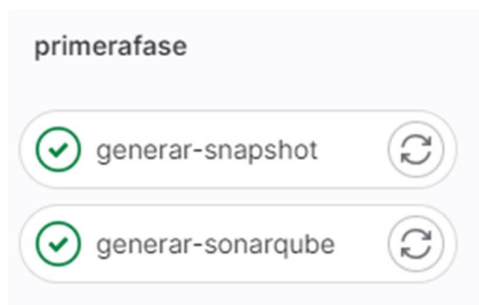
### 2.2.9. Ejecución de pipelines

Se configura el archivo “.gitlab-ci.yml”, que se encuentra en la raíz del proyecto. Al aceptarse los cambios (commits) realizados en el proyecto a través de un merge request automáticamente se ejecutará la primera fase.

La tarea de generar o compilar es la primera en ejecutar y la más importante ya que todas las tareas siguientes dependen de la ejecución correcta de la misma. Si la ejecución es incorrecta no avanza el flujo.



Si la ejecución es correcta la siguiente tarea a ejecutar es la revisión del código fuente con sonarqube.



Todas las fases dependen de la anterior tarea, en caso de existir un error en la ejecución el flujo se cancelaría.



Para la ejecución de las tareas se realizó la configuración de un corredor (gitlab-runners) interno utilizado únicamente para el proyecto

And this registration token:

6R1348941ouMxxPPUzQoDnQEUXfUz 

Reset registration token

Show runner installation instructions

## Assigned project runners

 #76 (ACTWKDB\_e) 

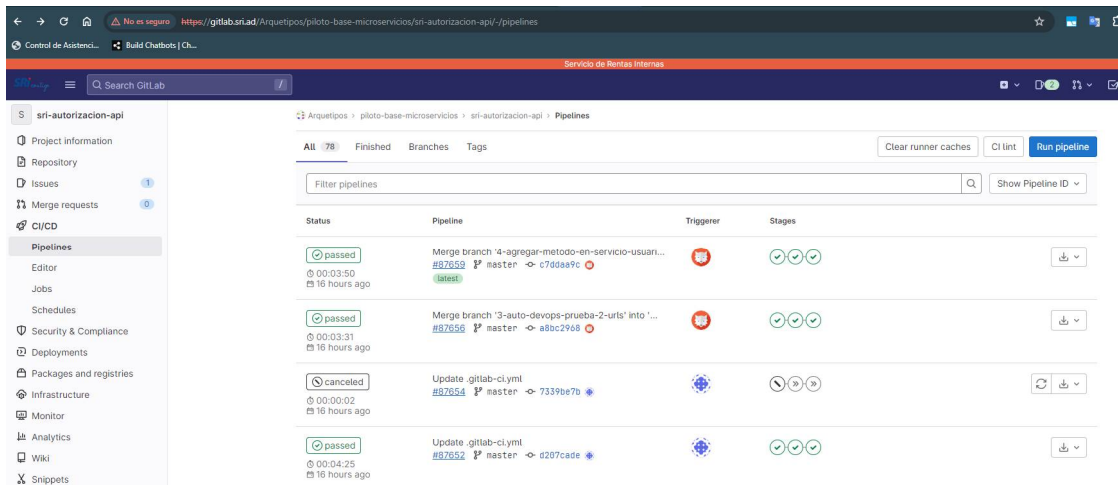
kubernetes

kube

  Remove runner

Las pruebas realizadas se pueden revisar en el siguiente link:

<https://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api/-/pipelines>



The screenshot shows the GitLab CI/CD interface for the project 'sri-autorizacion-api'. The left sidebar contains navigation links for Project Information, Repository, Issues, Merge requests, CI/CD, Pipelines, Editor, Jobs, Schedules, Security & Compliance, Deployments, Packages and registries, Infrastructure, Monitor, Analytics, Wiki, and Snippets. The main content area displays a list of pipelines with columns for Status, Pipeline, Triggerer, and Stages. The pipelines are filtered by 'All' (78) and show a list of recent runs with their status (passed, canceled, or failed) and the specific commit they were triggered by.

| Status   | Pipeline                                                                                      | Triggerer                                                                           | Stages |
|----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------|
| passed   | Merge branch '14-agregar-metodo-en-servicio-usuan...'<br>#87659 P master -> c7d0a9fc (latest) |  | ✓✓✓    |
| passed   | Merge branch '3-auto-devops-prueba-2-urls' into '...'<br>#87656 P master -> a8bc2968          |  | ✓✓✓    |
| canceled | Update .gitlab-ci.yml<br>#87654 P master -> 7339be7b                                          |  | ✗✗✗    |
| passed   | Update .gitlab-ci.yml<br>#87652 P master -> d207cade                                          |  | ✓✓✓    |

Arquetipos > piloto-base-microservicios > sri-autorizacion-api > Pipelines > #87659

passed Pipeline #87659 triggered 16 hours ago by Arreaga Desiderio, Christian Guillermo

## Merge branch '4-agregar-metodo-en-servicio-usuario' into 'master'

Resolve "Agregar método en servicio Usuario"

Closes #4

See merge request !4

🕒 5 jobs for `master` in 3 minutes and 50 seconds (queued for 2 seconds)

📄 `latest`

🔗 `c7ddaa9c`

🔗 No related merge requests found.

Pipeline Needs Jobs 5 Tests 0

primerafase

generar-snapshot

generar-sonarqube

segundafase

crear-imagen

tag-imagen

tercerafase

ejecutar-imagen

## 3. Resultados del Piloto

### 3.1. Resultados de la construcción del laboratorio del piloto

En la interacción de implementación del laboratorio del piloto se obtuvieron los siguientes resultados:

- Mediante la integración de nodo principal de GitLab con el servidor GitLab runner fue posible aislar el procesamiento del pipeline.
- El diseño del pipeline del flujo CI/CD permitió interactuar al GitLab runner con otros componentes como SonarQube, Nexus y el clúster de Kubernetes.
- Mediante la configuración del acceso a un registro de imágenes privado fue posible el almacenamiento y actualización de la imagen Docker que contiene el aplicativo de prueba.
- La implementación del laboratorio permitió identificar componentes adicionales necesarios para el funcionamiento del aplicativo de pruebas tales como VLAN's, certificados, configuraciones a nivel del balanceador de capa 3, configuraciones de redirección a nivel de red, configuraciones de servidores web.

### **3.2. Resultados del proceso CI/CD**

- El piloto demuestra que se puede habilitar el modelo de desarrollo que entregue valor al ritmo que exige el negocio.
- Al implementar el piloto se logrará que el desarrollador construya el código con seguridad desde el inicio del desarrollo.
- Los ciclos de desarrollo-pruebas y liberación-operación se aceleran con la automatización del proceso CI/CD, lo que aporta al cambio de cultura organizacional colocando en el centro al negocio.

## **4. Conclusiones**

### **4.1. De parte de Desarrollo**

- Al contar con un flujo de integración y despliegue automatizado, los tiempos de desarrollo se enmarcan en la codificación específica del requerimiento, ya que una vez que se haya realizado pruebas específicas en un ambiente local, se puede lanzar el cambio para continuar con su paso por certificación y producción de una forma dinámica y automática.
- Mantener un ambiente de desarrollo integrado con todos los sistemas dependientes, permite realizar pruebas de integración tanto automáticas como manuales, evitando desperdicio de tiempos de configuraciones de ambientes locales.
- El Departamento de Desarrollo al ser el contacto directo con el Usuario como receptor de cambios y requerimientos, al contar con la automatización de desarrollo y pruebas, es una gran ventaja ya que se tendrá un ambiente disponible desde el desarrollo temprano para que el usuario pueda realizar pruebas y así verificar los cambios aplicados e implementados en un ambiente no productivo, con lo cual pueden encontrar errores y fallas más temprano que con el desarrollo tradicional.
- En el camino de la ejecución de este piloto inicial para adoptar un flujo de trabajo de integración y entrega continua, nos hemos enfrentado algunos desafíos debido a la asignación de tareas propias del área de desarrollo con mayor prioridad, lo cual implica la falta de tiempo para investigación y aprendizaje, así como también los procedimientos burocráticos que se debe seguir para asignación de recursos, accesos y permisos, entre otros, han generado obstáculos en el desenvolvimiento normal del piloto.

### **4.2. De parte de Seguridad Informática**

- La automatización del proceso de desarrollo y liberaciones conlleva un desafío en el ámbito de seguridad informática.
- Se debe integrar soluciones de seguridad informática específicas para un proceso de DevSecOps, donde existan controles tales como el análisis de código, análisis de imágenes en los repositorios, gestión de vulnerabilidades, gestión de parches y gestión de credenciales de aplicaciones.
- Se debe automatizar los puntos de control de seguridad en todo el proceso, sin que esto ralentice en mayor medida el proceso de desarrollo de aplicaciones.

#### **4.2.1. De Control de Calidad**

- Los recursos en infraestructura para ambientes de pruebas son limitados.
- La automatización de pruebas contribuye a la mejora de la calidad del software contribuyendo consistencia, rápida retroalimentación, repetibilidad de ejecuciones, ahorro de tiempo y recursos, detección temprana de defectos y escalabilidad.
- Cuanto menor sea el tiempo que los desarrolladores dediquen a esperar los resultados de las pruebas, más tiempo tendrán para abordar las correcciones de errores y realizar cambios en el código.
- La combinación de CI/CD y pruebas automatizadas ayuda a reducir los tiempos de entrega, mejora la calidad del software al atrapar errores y proporciona una mayor confianza en los cambios que se implementan en producción. Este enfoque fomenta una cultura de desarrollo más ágil y eficiente.
- La adopción de una cultura DevSecOps en la Dirección Nacional de Tecnología del Servicio de Rentas Internas (SRI) como parte de un piloto inicial ha enfrentado desafíos significativos en los últimos tres meses. La lentitud en los procesos de atención de requerimientos, la limitada disponibilidad de ambientes de pruebas, la falta de tiempo para investigación y aprendizaje, así como la sobrecarga de actividades para los miembros del equipo, han generado obstáculos en el desenvolvimiento normal de la iniciativa. Estos factores han impactado de alguna forma en la capacidad de avanzar rápidamente en dicha prueba de concepto de la adopción de un flujo completo DevSecOps.

#### **4.3. De parte de Infraestructura y Operaciones**

- Se ha identificado que la asignación de tareas adicionales a cada uno de los elementos del equipo de trabajo con mayor prioridad ha generado obstáculos en la implementación del piloto.
- Para avanzar de manera ágil en la adopción del proceso DevSecOps en toda la Dirección de Tecnología, se destaca la necesidad de otorgar importancia y prioridad al tiempo asignado al equipo multidisciplinario.
- La lentitud en los procesos internos, la falta de capacidad, asignación y disponibilidad de ambientes de pruebas para el piloto, así como, la sobrecarga de actividades ha impactado negativamente en la capacidad de avanzar rápidamente en la prueba de concepto de la adopción de un flujo completo DevSecOps.

#### **4.4. De parte de Planificación TI**

- Se concluye que la arquitectura planteada fue posible implementarla mediante los recursos que estuvieron disponibles para el piloto.
- La implementación de CI/CD permitió el aprovechamiento de nuevas tecnologías y la visualización más rápida de los cambios ocurridos en la aplicación de ejemplo.
- La implementación de CI/CD necesita la colaboración de todas las áreas involucradas para lograr establecer un nuevo proceso de despliegue.
- Se concluye que es importante contar con el apoyo y soporte comprometido de todo el equipo a fin de evitar dilatar el tiempo de tareas necesarias para la conformación

de los ambientes tales como los permisos de firewall y configuraciones de red que jugaron un papel importante en este piloto y cuyo demorado tiempo de atención repercutieron en el alcance.

## **5. Recomendaciones**

### **5.1. De parte de Desarrollo**

- Crear el proceso DevOps, acorde a los lineamientos vigentes, para poder integrar a todos los departamentos de la Dirección Nacional de Tecnología y automatizar las tareas de integración y despliegue continuo.
- Buscar acelerar los ciclos de desarrollo, pruebas y puesta en producción de manera iterativa e incremental para todos los cambios requeridos y ofrecer valor a los usuarios.
- Aportar a los equipos prácticas ágiles y prácticas de DevOps, que permitan desarrollos y despliegues en plazos más cortos, facilitando la frecuencia de implementación, mejorando la cultura del equipo de desarrollo y aumentando la colaboración entre los departamentos de TI para mejorar la experiencia del usuario.
- Adoptar enfoques ágiles en el ámbito de todas las aplicaciones, en la modernización de integraciones y automatización de tareas, nos trae grandes beneficios en el desempeño y también para mejorar la seguridad, la escalabilidad, productividad y experiencia de usuario.
- Implementar el proceso DevSecOps no solo debe implicar el uso de la última tecnología, también debe incluir las modificaciones en los procesos de cambios, desarrollo, pruebas y liberaciones, ya que la clave es entregar valor verdadero a los usuarios, y esto se puede obtener con entregas frecuentes, con mayor seguridad y calidad.
- Para permitir avanzar de manera más ágil en la adopción del proceso DevSecOps se debe considerar la importancia y prioridad al tiempo asignado al equipo multidisciplinario, así como también la mejora y optimización de los procesos y procedimientos internos, de tal forma que no generen retrasos en la continuidad de las actividades ni desperdicios de tiempo por excesiva o innecesaria documentación o por actividades repetitivas o innecesarias.

### **5.2. De parte de Seguridad Informática**

- Se debe realizar un estudio de mercado sobre soluciones de seguridad informática que apoyen al proceso de DevOps con el objetivo de ver cuál es la tendencia del mercado actual y con el apoyo de los Fabricantes aterrizar la necesidad del SRI en cuanto a seguridad en este proceso.
- Se debe enfocar la seguridad en el análisis del código y gestión de vulnerabilidades desde la fase de desarrollo y corregir la mayor cantidad de vulnerabilidades o las más críticas desde esta fase para poder agilizar el flujo del proceso en las siguientes fases.
- La mayor parte de herramientas de seguridad para DevOps están orientadas a un ambiente en nube por lo que se debe considerar este particular.



### **5.3. De parte de Infraestructura y Operaciones**

- Se sugiere dimensionar y asignar un ambiente de pruebas para el siguiente piloto a ser ejecutado con la finalidad de optimizar el tiempo de ejecución.
- Se recomienda priorizar, en los diversos departamentos de la Dirección Nacional de Tecnología, la implementación de una cultura DevSecOps. Así como, extender la asignación de recursos asignados con el propósito de identificar de manera integral los desafíos tecnológicos inherentes a la adopción de esta metodología.
- Se sugiere realizar una identificación completa y transversal de las distintas tecnologías y herramientas empleadas a lo largo del ciclo de producción de aplicaciones. Este análisis debería abordar no solo sus funciones básicas, sino también destacar aquellas capacidades que puedan ser aprovechadas al máximo. El objetivo es generar definiciones claras de interacción y uso que beneficien a todos los departamentos de la Dirección Nacional de Tecnología.

### **5.4. De Control de Calidad**

- Estar al día con versiones actuales de las herramientas, ofrece mayor beneficio como el rendimiento y seguridad, nuevas características y funcionalidades.
- Priorizar la revisión y optimización de los procesos relacionados con la gestión de configuraciones de red y permisos en accesos a servidores. Esto puede implicar la automatización de ciertas tareas para acelerar los tiempos de respuesta, o a su vez dando la prioridad necesaria a este tipo de iniciativas.
- Gestionar la disponibilidad de ambientes de pruebas para la implementación de la arquitectura completa de la iniciativa. Esto podría incluir la creación de políticas para la asignación eficiente de recursos y la implementación de herramientas de aprovisionamiento rápido de entornos.
- Proporcionar tiempo y recursos específicos para la investigación, aprendizaje, configuración y puesta en marcha de los componentes clave de la entrega y despliegue continuos. Esto podría incluir capacitaciones, sesiones de trabajo dedicadas y acceso a expertos en la implementación de DevSecOps.
- Evaluar y redistribuir las cargas de trabajo entre los miembros del equipo. Esto podría implicar la reevaluación de prioridades y la asignación de recursos adicionales para evitar la sobrecarga de actividades y permitir la dedicación adecuada a la iniciativa DevSecOps.
- Mejorar la comunicación interna sobre las metas y expectativas realistas en cuanto a los plazos y los resultados esperados. Asegurarse de que todos los miembros del equipo comprendan la importancia de la iniciativa y estén alineados con los objetivos de la implementación de la cultura DevSecOps.
- Fomentar la colaboración entre los departamentos de la Dirección de Tecnología, reconociendo la interdependencia de las actividades y garantizando una sinergia efectiva para la implementación de una cultura DevSecOps.
- La Dirección Nacional de Tecnología del SRI puede superar los desafíos actuales y avanzar hacia una implementación exitosa de una cultura DevSecOps, asegurando así una transformación positiva en los procesos y la seguridad de los servicios tecnológicos.

### **5.5. De Planificación TI**

- Se recomienda la planificación de nuevos pilotos mediante los cuales se pueda seguir trabajando en la iniciativa de DevSecOps.
- Se recomienda que se incremente el porcentaje de asignación de los miembros del equipo y se asigne una priorización más alta a la atención de las tareas relacionadas con la iniciativa DevSecOps.
- Se recomienda se brinde la importancia y el apoyo oportuno necesario para la configuración de ambientes en tareas como la asignación de permisos de firewall de igual manera en la ejecución de investigaciones y asistencia de los miembros del equipo.
- Con la finalidad de socializar y hacer partícipes a todas las áreas de la Dirección Nacional de Tecnología se recomienda extender la asignación por lo menos con interacciones a demanda a áreas tales como Redes, Aplicaciones, Planificación TI entre otras.
- Se recomienda evaluar la asignación de todos recursos físicos necesarios para los siguientes pilotos de la iniciativa DevSecOps.

## 6. Glosario

**API.-** Application Program Interface

**CI/CD.-** Continuous Integration / Continuous Deployment; Integración Continua / Despliegue Continuo

**DevOps.-** Acrónimo inglés de development (Desarrollo) y operations (operaciones)

**JDK.-** Java Development Kit

**JVM.-** Java Virtual Machine

**On-Premise.-** En instalaciones propias

**REST.-** Representational State Transfer

**SaaS.-** Software as a Service

**SO.-** Sistema Operativo

**SSH.-** Secure Shell

**VM.-** Virtual Machine (Máquina Virtual)

**VLAN.-**Virtual LAN

**YAML.-** Del acrónimo inglés Yet Another Markup Language (Otro lenguaje de marcado más)

## 7. Bibliografía

1. Quarkus (2023), Guides - Latest: Tutorials Short and focused exercises to get you going quickly, <https://quarkus.io/guides/> .
2. Eclipse EE4j (2023), The Jakarta EE Tutorial, <https://eclipse-ee4j.github.io/jakartaee-tutorial/>
3. *gitlab.com*. (2023). Obtenido de GitLab: <https://about.gitlab.com/platform/>
4. Kubernetes (2023), Kubernetes Documentation, <https://kubernetes.io/docs/home/>
5. Red Hat (2023), La integración y la distribución continuas (CI/CD), <https://www.redhat.com/es/topics/devops/what-is-ci-cd>